ArrayList e Regras de Negócio

Projeto: Sistema de Projetos e Portfólio
Pré-requisito: Aula 2 — Classes e Objetos

Nesta aula, o aluno deixa de trabalhar com objetos isolados e passa a gerenciar **vários projetos ao mesmo tempo**, utilizando `ArrayList`. Também começaremos a separar responsabilidades no sistema, criando a primeira classe de regras de negócio: `ProjetoService`.

---

1. Objetivos da aula

Ao final da aula, o aluno deverá ser capaz de:

- compreender o conceito de coleção em Java;
- utilizar `ArrayList`;
- armazenar vários objetos de uma mesma classe;
- adicionar elementos à lista;
- percorrer uma lista;
- consultar objetos;
- buscar um projeto pelo ID;
- filtrar projetos por categoria;
- filtrar projetos por status;
- remover um projeto;
- compreender o papel da camada de serviço;
- separar a classe de dados da classe de regras de negócio.

2. Retomando a Aula 2

Na aula anterior criamos a classe:

```
Projeto
```

E conseguimos criar objetos:

```
Projeto projeto1 = new Projeto(
    1,
    "Sistema Acadêmico",
    "Sistema para gerenciamento acadêmico",
    "Software",
    "Em desenvolvimento"
);

Projeto projeto2 = new Projeto(
    2,
    "Site Institucional",
    "Website institucional",
    "Web",
    "Concluído"
);
```

O problema é que, em um sistema real, teremos muitos projetos.

Não queremos criar:

```
Projeto projeto1;
Projeto projeto2;
Projeto projeto3;
Projeto projeto4;
Projeto projeto5;
Projeto projeto6;
```

Precisamos de uma estrutura capaz de armazenar uma quantidade variável de objetos.

É aí que entra o `ArrayList`.

3. O que é uma coleção?

Uma **coleção** é uma estrutura utilizada para armazenar vários elementos.

```
Lista de Projetos

[Projeto 1]
[Projeto 2]
[Projeto 3]
[Projeto 4]
[Projeto 5]
```

Em Java existem várias estruturas de coleção. Nesta aula utilizaremos:

```
ArrayList
```

---

4. Criando um ArrayList simples

Comece com um exemplo pequeno, separado do projeto principal.

```java
import java.util.ArrayList;

public class ExemploLista {

    public static void main(String[] args) {

        ArrayList<String> nomes = new ArrayList<>();

        nomes.add("Ana");
        nomes.add("Carlos");
        nomes.add("Mariana");

        System.out.println(nomes);

    }
}
```

Resultado:

```
[Ana, Carlos, Mariana]
```

Explique:

```
ArrayList<String>
```

significa:

> Esta lista armazenará objetos do tipo `String`.

---

5. Adicionando elementos

Utilizamos:

```
add()
```

Exemplo:

```
nomes.add("José");
```

Agora:

```
Ana
Carlos
Mariana
José
```

---

6. Descobrindo quantos elementos existem

Utilizamos:

```
size()
```

Exemplo:

```
System.out.println(nomes.size());
```

Resultado:

```
4
```

7. Acessando um elemento

Podemos utilizar:

```
get()
```

Exemplo:

```
System.out.println(nomes.get(0));
```

Resultado:

```
Ana
```

Como os índices começam em 0:

```
Índice 0 → Ana
Índice 1 → Carlos
Índice 2 → Mariana
Índice 3 → José
```

---

8. Removendo um elemento

Podemos utilizar:

```
remove()
```

Exemplo:

```
nomes.remove(1);
```

O elemento de índice 1 será removido.

9. ArrayList de objetos

Agora chegamos ao ponto principal da aula. Podemos criar uma lista de objetos `Projeto`.

```
import java.util.ArrayList;
import model.Projeto;

public class Main {

    public static void main(String[] args) {

        ArrayList<Projeto> projetos = new ArrayList<>();

    }
}
```

Explique:

```
ArrayList<Projeto>
```

Significa que esta lista armazenará somente objetos da classe `Projeto`.

---

10. Adicionando projetos na lista

Primeiro criamos os objetos:

```
Projeto projeto1 = new Projeto(
    1,
    "Sistema Acadêmico",
    "Sistema para gerenciamento acadêmico",
    "Software",
    "Em desenvolvimento"
);

Projeto projeto2 = new Projeto(
    2,
    "Site Institucional",
    "Website de uma instituição",
    "Web",
    "Concluído"
);
```

Depois:

```
projetos.add(projeto1);
projetos.add(projeto2);
```

Temos agora:

```
projetos
│
├── Projeto 1
└── Projeto 2
```

---

11. Percorrendo a lista

Uma das maneiras mais simples é o `for-each`.

```
for (Projeto projeto : projetos) {

    System.out.println(projeto.getNome());

}
```

Resultado:

```
Sistema Acadêmico
Site Institucional
```

Explique a leitura:

```
for (Projeto projeto : projetos)
```

como:

> "Para cada objeto `Projeto` existente dentro da lista `projetos`, coloque temporariamente esse objeto na variável `projeto`."

12. Exibindo todos os dados

Podemos usar o método criado na Aula 2:

```
for (Projeto projeto : projetos) {

    projeto.exibirDados();

    System.out.println("--------------------");
}
```

Resultado:

```
ID: 1
Nome: Sistema Acadêmico
Descrição: Sistema para gerenciamento acadêmico
Categoria: Software
Status: Em desenvolvimento
--------------------
ID: 2
Nome: Site Institucional
Descrição: Website de uma instituição
Categoria: Web
Status: Concluído
--------------------
```

---

13. Um problema de organização

Até aqui colocamos tudo dentro do `Main`.

Por exemplo:

```
ArrayList<Projeto> projetos = new ArrayList<>();

projetos.add(projeto1);

for (Projeto projeto : projetos) {
    System.out.println(projeto.getNome());
}
```

Mas, conforme o sistema crescer, teremos código para:

- adicionar projeto;
- listar projeto;
- buscar projeto;
- alterar projeto;
- remover projeto;
- pesquisar por categoria;
- pesquisar por status.

Se tudo ficar no `Main`, o programa ficará difícil de manter. Por isso criaremos:

```
ProjetoService
```

---

14. O que é uma classe Service?

A classe `Service` será responsável pelas regras e operações relacionadas aos projetos.

Teremos:

```
Projeto.java
    │
    │ representa os dados
    ▼

ProjetoService.java
    │
    │ gerencia os projetos
    ▼

Main.java
    │
    │ testa o sistema
    ▼
```

Podemos pensar assim:

```
Projeto
= "O que é um projeto?"

ProjetoService
= "O que podemos fazer com os projetos?"
```

15. Criando o pacote `service`

A estrutura passa a ser:

```
SistemaProjetos/
│
└── src/
    │
    ├── Main.java
    │
    ├── model/
    │   └── Projeto.java
    │
    └── service/
        └── ProjetoService.java
```

---

16. Criando `ProjetoService`

Crie:

```java
package service;

import java.util.ArrayList;
import java.util.List;

import model.Projeto;

public class ProjetoService {

    private List<Projeto> projetos;

    public ProjetoService() {

        projetos = new ArrayList<>();

    }

}
```

Explique:

```
private List<Projeto> projetos;
```

Estamos declarando uma lista que armazena objetos `Projeto`. E:

```
projetos = new ArrayList<>();
```

criamos efetivamente a lista.

---

17. Por que usar `List` de um lado e `ArrayList` do outro?

Observe:

```
private List<Projeto> projetos;
```

e:

```
projetos = new ArrayList<>();
```

Explique de forma simples:

> `List` representa o tipo mais geral da lista, enquanto `ArrayList` é uma implementação concreta.

Para este momento, não é necessário aprofundar interfaces.

O aluno pode compreender apenas:

```
List
   ↓
ArrayList
```

e utilizar esse padrão durante o curso.

18. Método adicionar

Agora criaremos o primeiro método da classe de serviço.

```
public void adicionar(Projeto projeto) {

    projetos.add(projeto);

}
```

A classe fica:

```
package service;

import java.util.ArrayList;
import java.util.List;

import model.Projeto;

public class ProjetoService {

    private List<Projeto> projetos;

    public ProjetoService() {
        projetos = new ArrayList<>();
    }

    public void adicionar(Projeto projeto) {
        projetos.add(projeto);
    }

}
```

19. Testando o método adicionar

No `Main.java`:

```
import model.Projeto;
import service.ProjetoService;

public class Main {

    public static void main(String[] args) {

        ProjetoService service = new ProjetoService();

        Projeto p1 = new Projeto(
            1,
            "Sistema Acadêmico",
            "Sistema para gerenciamento acadêmico",
            "Software",
            "Em desenvolvimento"
        );

        service.adicionar(p1);

    }
}
```

Agora o objeto `Projeto` não é colocado diretamente na lista pelo `Main`.

Quem gerencia a lista é:

```
ProjetoService
```

---

20. Método listar

Adicione:

```
public List<Projeto> listar() {

    return projetos;

}
```

Agora:

```
for (Projeto projeto : service.listar()) {

    projeto.exibirDados();

}
```

---

21. Criando três projetos

No `Main`:

```
Projeto p1 = new Projeto(
    1,
    "Sistema Acadêmico",
    "Sistema para gerenciamento acadêmico",
    "Software",
    "Em desenvolvimento"
);

Projeto p2 = new Projeto(
    2,
    "Site Institucional",
    "Website institucional",
    "Web",
    "Concluído"
);

Projeto p3 = new Projeto(
    3,
    "Aplicativo Mobile",
    "Aplicativo de serviços",
    "Mobile",
    "Planejado"
);
```

Depois:

```
service.adicionar(p1);
service.adicionar(p2);
service.adicionar(p3);
```

22. Exibindo a quantidade de projetos

Podemos fazer:

```
System.out.println(
    "Total de projetos: " + service.listar().size()
);
```

Resultado:

```
Total de projetos: 3
```

---

23. Buscar projeto pelo ID

Agora começamos a criar uma regra de negócio.

Precisamos localizar um projeto específico. Crie:

```
public Projeto buscarPorId(int id) {

    for (Projeto projeto : projetos) {

        if (projeto.getId() == id) {

            return projeto;

        }
    }

    return null;
}
```

Explique passo a passo.

Primeiro:

```
for (Projeto projeto : projetos)
```

percorremos todos os projetos.

Depois:

```
if (projeto.getId() == id)
```

verificamos se o ID corresponde ao procurado. Se encontrarmos:

```
return projeto;
```

Se terminarmos a busca sem encontrar:

```
return null;
```

---

24. Testando a busca pelo ID

No `Main`:

```
Projeto encontrado = service.buscarPorId(2);
```

Depois:

```
if (encontrado != null) {

    System.out.println("Projeto encontrado:");

    encontrado.exibirDados();

} else {

    System.out.println("Projeto não encontrado.");

}
```

Resultado:

```
Projeto encontrado:

ID: 2
Nome: Site Institucional
Descrição: Website institucional
Categoria: Web
Status: Concluído
```

25. Buscar projetos por categoria

Agora criaremos um filtro.

```
public List<Projeto> buscarPorCategoria(String categoria) {

    List<Projeto> resultado = new ArrayList<>();

    for (Projeto projeto : projetos) {

        if (projeto.getCategoria()
                   .equalsIgnoreCase(categoria)
        ) {
            resultado.add(projeto);
        }
    }

    return resultado;
}
```

---

26. Entendendo o filtro

Primeiro criamos uma lista vazia:

```
List<Projeto> resultado = new ArrayList<>();
```

Percorremos os projetos:

```
for (Projeto projeto : projetos)
```

Comparamos:

```
projeto.getCategoria().equalsIgnoreCase(categoria)
```

Se for igual:

```
resultado.add(projeto);
```

No final:

```
return resultado;
```

27. Por que `equalsIgnoreCase()`?

Imagine:

```
Web
web
WEB
Web
```

Com:

```
equalsIgnoreCase()
```

todos podem ser considerados iguais. Exemplo:

```
"Web".equalsIgnoreCase("WEB")
```

Resultado:

```
true
```

---

28. Testando a categoria

No `Main`:

```
System.out.println("PROJETOS WEB");

for (
    Projeto projeto :
    service.buscarPorCategoria("Web")
) {

    projeto.exibirDados();

}
```

29. Buscar por status

O raciocínio é praticamente o mesmo.

```
public List<Projeto> buscarPorStatus(String status) {

    List<Projeto> resultado = new ArrayList<>();

    for (Projeto projeto : projetos) {

        if (
            projeto.getStatus()
                   .equalsIgnoreCase(status)
        ) {

            resultado.add(projeto);

        }
    }

    return resultado;
}
```

---

30. Testando o status

```
System.out.println("PROJETOS CONCLUÍDOS");

for (
    Projeto projeto :
    service.buscarPorStatus("Concluído")
) {

    projeto.exibirDados();

}
```

31. Remover projeto

Agora podemos introduzir a exclusão.

Uma implementação didática:

```
public boolean removerPorId(int id) {

    Projeto projeto = buscarPorId(id);

    if (projeto != null) {

        projetos.remove(projeto);

        return true;
    }

    return false;
}
```

Observe que reaproveitamos:

```
buscarPorId(id)
```

Isso é importante.

> Em vez de duplicarmos lógica, reutilizamos um método que já existe.

32. Testando a remoção

```
boolean removido = service.removerPorId(2);

if (removido) {

    System.out.println(
        "Projeto removido com sucesso."
    );

} else {

    System.out.println(
        "Projeto não encontrado."
    );
}
Depois:
System.out.println(
    "Total: " + service.listar().size()
);
```

---

33. Regra: não permitir ID duplicado

Agora podemos introduzir uma regra real.

Se adicionarmos:

```
Projeto p1 = new Projeto(1, ...);
Projeto p2 = new Projeto(1, ...);
```

teremos dois projetos com o mesmo ID.

Isso não deveria acontecer.

Modifique:

```
public boolean adicionar(Projeto projeto) {

    Projeto existente =
        buscarPorId(projeto.getId());

    if (existente != null) {

        return false;

    }

    projetos.add(projeto);

    return true;
}
```

Agora o método retorna:

```
true
```

quando adiciona. E:

```
false
```

quando encontra ID duplicado.

34. Testando a regra

```
boolean cadastrado = service.adicionar(p1);

if (cadastrado) {

    System.out.println(
        "Projeto cadastrado."
    );

} else {

    System.out.println(
        "Já existe um projeto com esse ID."
    );

}
```

Isso é um excelente momento para explicar:

> **Regra de negócio é uma regra que representa como o sistema deve funcionar.**

Neste caso:

> "Não podem existir dois projetos com o mesmo ID."

35. Outra regra de negócio: nome obrigatório

Podemos ampliar:

```
public boolean adicionar(Projeto projeto) {

    if (
        projeto.getNome() == null ||
        projeto.getNome().isBlank()
    ) {

        return false;

    }

    if (
        buscarPorId(projeto.getId()) != null
    ) {

        return false;

    }

    projetos.add(projeto);

    return true;
}
```

Agora temos duas regras:

ID não pode ser repetido.

Nome não pode estar vazio.

36. Código completo de `ProjetoService.java`

Ao final da aula:

```java
package service;

import java.util.ArrayList;
import java.util.List;

import model.Projeto;

public class ProjetoService {

    private List<Projeto> projetos;

    public ProjetoService() {

        projetos = new ArrayList<>();

    }

    public boolean adicionar(Projeto projeto) {

        if (
            projeto.getNome() == null ||
            projeto.getNome().isBlank()
        ) {

            return false;

        }

        if (
            buscarPorId(projeto.getId()) != null
        ) {

            return false;

        }

        projetos.add(projeto);

        return true;
    }
```

```
    public List<Projeto> listar() {

        return projetos;

    }

    public Projeto buscarPorId(int id) {

        for (Projeto projeto : projetos) {

            if (projeto.getId() == id) {

                return projeto;

            }
        }

        return null;
    }

    public List<Projeto> buscarPorCategoria(
        String categoria
    ) {

        List<Projeto> resultado =
            new ArrayList<>();

        for (Projeto projeto : projetos) {

            if (
                projeto.getCategoria()
                        .equalsIgnoreCase(categoria)
            ) {

                resultado.add(projeto);

            }
        }

        return resultado;
    }

    public List<Projeto> buscarPorStatus(
        String status
    ) {
```

```
        List<Projeto> resultado =
            new ArrayList<>();

        for (Projeto projeto : projetos) {

            if (
                projeto.getStatus()
                       .equalsIgnoreCase(status)
            ) {

                resultado.add(projeto);

            }
        }

        return resultado;
    }

    public boolean removerPorId(int id) {

        Projeto projeto = buscarPorId(id);

        if (projeto != null) {

            projetos.remove(projeto);

            return true;

        }

        return false;
    }
}
```

37. `Main.java` completo para teste

```
import model.Projeto;
import service.ProjetoService;

public class Main {

    public static void main(String[] args) {

        ProjetoService service =
            new ProjetoService();

        Projeto p1 = new Projeto(
            1,
            "Sistema Acadêmico",
            "Sistema para gerenciamento acadêmico",
            "Software",
            "Em desenvolvimento"
        );

        Projeto p2 = new Projeto(
            2,
            "Site Institucional",
            "Website institucional",
            "Web",
            "Concluído"
        );

        Projeto p3 = new Projeto(
            3,
            "Aplicativo Mobile",
            "Aplicativo para prestação de serviços",
            "Mobile",
            "Planejado"
        );

        service.adicionar(p1);
        service.adicionar(p2);
        service.adicionar(p3);

        System.out.println(
            "TOTAL DE PROJETOS: "
            + service.listar().size()
        );
```

```
        System.out.println();

        System.out.println("LISTA DE PROJETOS");

        for (Projeto projeto : service.listar()) {
            projeto.exibirDados();
            System.out.println("----------------");

        }

        System.out.println();

        System.out.println("BUSCA PELO ID 2");

        Projeto encontrado =
            service.buscarPorId(2);

        if (encontrado != null) {
            encontrado.exibirDados();

        }

        System.out.println();

        System.out.println("PROJETOS WEB");

        for (Projeto projeto :
            service.buscarPorCategoria("Web")
        ) {

            projeto.exibirDados();

        }

        System.out.println();

        System.out.println("PROJETOS CONCLUÍDOS");

        for (Projeto projeto :
            service.buscarPorStatus("Concluído")
        ) {
            projeto.exibirDados();

        }
    }
}
```

38. Resultado esperado

Algo semelhante a:

```
TOTAL DE PROJETOS: 3

LISTA DE PROJETOS

ID: 1
Nome: Sistema Acadêmico
Descrição: Sistema para gerenciamento acadêmico
Categoria: Software
Status: Em desenvolvimento
----------------

ID: 2
Nome: Site Institucional
Descrição: Website institucional
Categoria: Web
Status: Concluído
----------------

ID: 3
Nome: Aplicativo Mobile
Descrição: Aplicativo para prestação de serviços
Categoria: Mobile
Status: Planejado
----------------

BUSCA PELO ID 2

ID: 2
Nome: Site Institucional
Descrição: Website institucional
Categoria: Web
Status: Concluído

PROJETOS WEB

ID: 2
Nome: Site Institucional
Descrição: Website institucional
Categoria: Web
Status: Concluído
```

```
PROJETOS CONCLUÍDOS

ID: 2
Nome: Site Institucional
Descrição: Website institucional
Categoria: Web
Status: Concluído
```

---

39. Conceito importante: separação de responsabilidades

Agora podemos mostrar:

```
Projeto.java
|
| guarda os dados
|
▼
ProjetoService.java
|
| aplica regras de negócio
|
▼
Main.java
|
| inicia e testa o sistema
|
▼
```

Projeto

Representa o objeto.

ProjetoService

Gerencia os projetos.

Main

Coordena/testa a execução.

Essa separação será extremamente importante quando chegarmos à API.

40. Preparando para a arquitetura futura

A estrutura começará a evoluir para:

```
WEB
 │
 ▼
API
 │
 ▼
ProjetoService
 │
 ▼
ProjetoCSV
```

E:

```
DESKTOP
 │
 ▼
API
 │
 ▼
ProjetoService
 │
 ▼
ProjetoCSV
```

A classe criada hoje:

```
ProjetoService
```

continuará sendo utilizada quando criarmos a API REST.

41. Atividade prática

Situação

O sistema possui os seguintes projetos:

```
1 -- Sistema Acadêmico -- Software -- Em desenvolvimento

2 -- Site Institucional -- Web -- Concluído

3 -- Aplicativo Mobile -- Mobile -- Planejado

4 -- Portal de Eventos -- Web -- Em desenvolvimento

5 -- Sistema Financeiro -- Software -- Concluído
```

O aluno deverá:

1. cadastrar os cinco projetos;
2. listar todos;
3. exibir a quantidade total;
4. buscar o projeto de ID `3`;
5. listar apenas projetos da categoria `Web`;
6. listar apenas projetos com status `Concluído`;
7. remover o projeto de ID `4`;
8. tentar cadastrar outro projeto com ID `1`;
9. verificar se a regra de ID duplicado funciona.

---

42. Desafio 1 — Contar projetos por categoria

Crie:

```
public int contarPorCategoria(
    String categoria
) {

    return buscarPorCategoria(categoria).size();

}
```

Teste:

```
System.out.println(
    "Projetos Web: "
    + service.contarPorCategoria("Web")
);
```

43. Desafio 2 — Alterar status

Crie na `ProjetoService`:

```
public boolean alterarStatus(
    int id,
    String novoStatus
) {

    Projeto projeto = buscarPorId(id);

    if (projeto == null) {

        return false;

    }

    projeto.setStatus(novoStatus);

    return true;
}
```

Teste:

```
service.alterarStatus(
    1,
    "Concluído"
);
```

Depois:

```
Projeto projeto =
    service.buscarPorId(1);

System.out.println(
    projeto.getStatus()
);
```

Resultado:

```
Concluído
```

44. Desafio 3 — Buscar pelo nome

Uma possibilidade:

```
public List<Projeto> buscarPorNome(
    String texto
) {

    List<Projeto> resultado =
        new ArrayList<>();

    for (Projeto projeto : projetos) {

        if (
            projeto.getNome()
                   .toLowerCase()
                   .contains(
                       texto.toLowerCase()
                   )
        ) {

            resultado.add(projeto);

        }
    }

    return resultado;
}
```

Agora:

```
service.buscarPorNome("sistema");
```

poderia retornar:

```
Sistema Acadêmico
Sistema Financeiro
```

Esse método será interessante futuramente para o **campo de pesquisa da interface Web**.

45. Perguntas para fechamento

1. O que é um `ArrayList`?

Uma lista dinâmica utilizada para armazenar vários elementos.

2. O que significa:

List<Projeto>

Uma lista que armazena objetos da classe `Projeto`.

3. Para que serve `add()`?

Adicionar um elemento.

4. Para que serve `size()`?

Informar a quantidade de elementos.

5. Para que serve o `for-each`?

Percorrer os elementos de uma coleção.

6. Qual o papel da `ProjetoService`?

Centralizar operações e regras relacionadas aos projetos.

7. Por que não colocar tudo no `Main`?

Porque isso dificulta organização, manutenção e reutilização do código.

46. Estrutura final do projeto

Ao final da Aula 3:

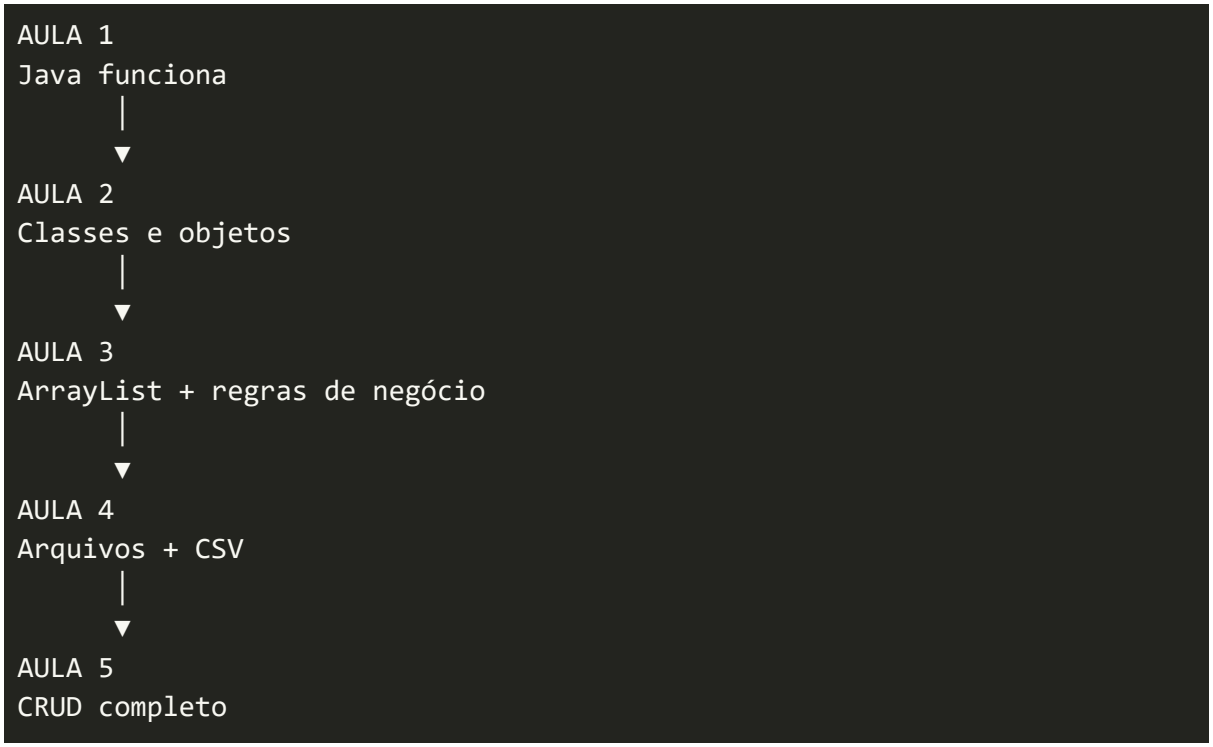
```
SistemaProjetos/
│
└── src/
    │
    ├── Main.java
    │
    ├── model/
    │   └── Projeto.java
    │
    └── service/
        └── ProjetoService.java
```

O sistema já possui:

```
Projeto
   │
   ▼
objeto

ProjetoService
   │
   ├── adicionar
   ├── listar
   ├── buscarPorId
   ├── buscarPorCategoria
   ├── buscarPorStatus
   └── removerPorId
```

47. O que construímos até agora?



Resultado esperado da Aula 3

Ao final desta aula, o aluno deverá conseguir dizer:

> **"Agora consigo armazenar vários projetos, percorrer a lista, pesquisar informações e aplicar regras de negócio."**

A partir da **Aula 4**, o principal problema será outro: atualmente, quando encerramos o programa, todos os projetos desaparecem. A próxima etapa será criar a **persistência em arquivo CSV**, fazendo com que os dados possam ser salvos e recuperados.